

ADAPTER APLICADO EN CALCULADORA

Patrones de Diseño: *Adapter*

Adaptación de tipos de datos de entrada
para operaciones aritméticas

Autor: **Angie Nathaly Cisneros Guerrero**

20242020084

Fecha: Septiembre 2026

Índice

1. Diagrama UML	2
2. Explicación de las Clases	3
2.1. Interfaces y Componentes Base	3
2.1.1. Interfaz Target	3
2.1.2. Clase Cliente	3
2.2. Patrón Adapter - Adaptador y Adaptee	5
2.2.1. Clase Adaptee	5
2.2.2. Clase Adaptador	5
3. Código de Cliente	7
3.1. Clase Main	7
4. Pruebas y Ejecución	9
4.1. Prueba de Ejecución	9
5. Repositorio	10
5.1. Enlace GitHub	10
5.2. Estructura de Paquetes	10

Diagrama UML

Descripción General

El proyecto implementa el patrón de diseño **Adapter** para compatibilizar la entrada de datos, la cual se lee como texto (**String**), y adaptarla a un formato numérico entero (**int**) requerido por el sistema para ejecutar las operaciones aritméticas de la calculadora.

Diagrama de Clases

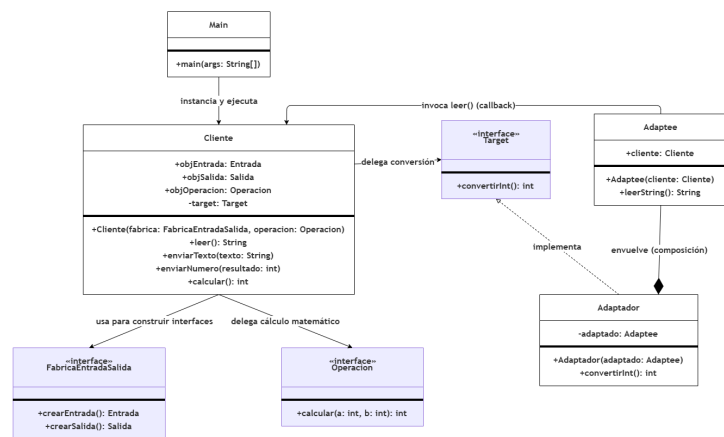


Figura 1: Diagrama de Clases del Sistema.

Explicación de las Clases

Interfaces y Componentes Base

2.1.1. Interfaz Target

Descripción

Define la interfaz que el cliente espera utilizar. En este caso, establece el contrato para convertir y obtener datos en formato de número entero (int).

Código Fuente

```
1 package calculadora.adapter;  
2  
3 /**  
4  *  
5  * @author natha  
6  */  
7 public interface Target {  
8  
9     public int convertirInt();  
10  
11 }
```

2.1.2. Clase Cliente

Descripción

Es la clase central que orquesta la ejecución. Consume entradas y salidas a través de sus fábricas y utiliza el **Target** para procesar las entradas antes de enviarlas a la operación matemática correspondiente. Internamente enlaza el adaptado (**Adaptee**) con su Adaptador.

Código Fuente

```
1 package calculadora.adapter;  
2  
3 /*  
4  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/  
5  * license-default.txt to change this license  
6  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/  
7  * Interface.java to edit this template  
8  */  
9  
10 import calculadora.adapter.Calculadora.FabricaEntradaSalida;  
11 import calculadora.adapter.Calculadora.Operation;  
12 import calculadora.adapter.Calculadora.Entrada;  
13 import calculadora.adapter.Calculadora.Salida;  
14  
15 /**  
16  *  
17  */
```

```
16  * @author
17  */
18  public class Cliente {
19
20      public Entrada objEntrada;
21      public Salida objSalida;
22      public Operacion objOperacion;
23      private Target target;
24
25      public Cliente(FabricaEntradaSalida fabrica, Operacion
26          operacion) {
27          this.objEntrada = (Entrada) fabrica.crearEntrada();
28          this.objSalida = (Salida) fabrica.crearSalida();
29          this.objOperacion = operacion;
30
31          Adaptee adaptee = new Adaptee(this);
32          this.target = new Adaptador(adaptee);
33      }
34
35      public String leer(){
36          enviarTexto("\u00D9Digite\u00D9un\u00D9numero");
37          return objEntrada.leer();
38      }
39
40      public void enviarTexto(String texto){
41          objSalida.salidaTexto(texto);
42      }
43      public void enviarNumero(int resultado){
44          objSalida.salidaNumero(resultado);
45      }
46
47      public int calcular (){
48          int a = target.convertirInt();
49          int b = target.convertirInt();
50          return objOperacion.calcular(a, b);
51      }
52  }
```

Patrón Adapter - Adaptador y Adaptee

Descripción

Se implementa el núcleo del patrón mediante dos clases que conectan el formato de la interfaz (Texto) con el cálculo (Enteros).

2.2.1. Clase Adaptee

Descripción

Contiene la lógica preexistente o base que necesita ser adaptada. Su función `leerString()` obtiene los datos desde el cliente en forma de cadena de texto.

Código Fuente

```
1 package calculadora.adapter;
2
3 /**
4  *
5  * @author natha
6  */
7 public class Adaptee {
8
9
10     public Cliente cliente;
11
12     public Adaptee(Cliente cliente) {
13         this.cliente = cliente;
14     }
15
16     public String leerString(){
17         return cliente.leer();
18     }
19
20
21 }
```

2.2.2. Clase Adaptador

Descripción

Clase que implementa **Target** y se compone del **Adaptee**. Transforma la incompatibilidad llamando al método del adaptado y ejecutando la conversión a enteros.

Código Fuente

```
1  /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/
   license-default.txt to change this license
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.
   java to edit this template
4  */
5  package calculadora.adapter;
6
```

```
7  /**
8   *
9   * @author natha
10  */
11 public class Adaptador implements Target{
12
13     private Adaptee adaptado;
14
15     public Adaptador(Adaptee adaptado) {
16         this.adaptado = adaptado;
17     }
18
19
20
21     @Override
22     public int convertirInt() {
23
24         String entrada = adaptado.leerString();
25         int entradaInt = Integer.parseInt(entrada);
26         return entradaInt;
27     }
28
29
30 }
```

Explicación

- **Intermediario:** En su constructor recibe un objeto Adaptee.
- **Traducción de Datos:** En `convertirInt()`, obtiene el String original a través de `adaptado.leerString()` y aplica `Integer.parseInt()` para devolver el int que el sistema necesita.

Código de Cliente

Clase Main

Código Fuente

```
1  /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/
   license-default.txt to change this license
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Main.
   java to edit this template
4  */
5  package calculadora.adapter;
6
7  import calculadora.adapter.Calculadora.FabricaSwing;
8  import calculadora.adapter.Calculadora.FabricaConsola;
9  import calculadora.adapter.Calculadora.FabricaArchivo;
10 import calculadora.adapter.Calculadora.FabricaEntradaSalida;
11 import calculadora.adapter.Calculadora.Multiplicacion;
12 import calculadora.adapter.Calculadora.Operacion;
13 import calculadora.adapter.Calculadora.Suma;
14
15 import javax.swing.JOptionPane;
16
17 /**
18  *
19  * @author natha
20  */
21 public class Main {
22
23     public static void main(String[] args) {
24
25         String opcion = JOptionPane.showInputDialog(
26             " Qu  interfaz desea usar digite el n  ?\n1.
27             Consola\n2. Swing\n3. Archivo "
28         );
29         String calculo = JOptionPane.showInputDialog(
30             " Qu  operacion desea hacer digite el n  ?\n1.
31             Suma\n2. Multiplicacion "
32         );
33
34         FabricaEntradaSalida fabrica;
35         Operacion objOperacion;
36         if ("1".equals(opcion)){
37             fabrica = new FabricaConsola();
38         }
39         else if ("2".equals(opcion)) {
40             fabrica = new FabricaSwing();
41         }
42         else{
43             fabrica = new FabricaArchivo();
44         }
```

```
43
44     if ("1".equals(calculo)){
45         objOperacion = new Suma();
46     }
47     else {
48         objOperacion = new Multiplicacion();
49     }
50
51     Cliente cliente = new Cliente(fabrica, objOperacion);
52     cliente.enviarTexto("└-----┘Calculadora└-----┘");
53     int resultado = cliente.calcular();
54     cliente.enviarTexto("└Resultado:");
55     cliente.enviarNumero(resultado);
56 }
57 }
```

Flujo de Ejecución

1. Se utiliza `JOptionPane` para solicitar al usuario el tipo de interfaz y la operación deseada.
2. Se instancia la familia de clases a través de la fábrica correspondiente (`Consola`, `Swing` o `Archivo`) y la `Operacion` requerida.
3. Se inicializa el `Cliente` con estos objetos. Al hacerlo, el cliente configura internamente el **Adaptador** conectándolo consigo mismo (su rol de `Adaptee`).
4. Se ejecuta el cálculo. El cliente pide el dato, el **Adaptador** se lo solicita como `String` (mediante `Adaptee`) y lo convierte automáticamente a `int`, permitiendo realizar la suma o multiplicación exitosamente.

Pruebas y Ejecución

Prueba de Ejecución



Figura 2: Salida 1

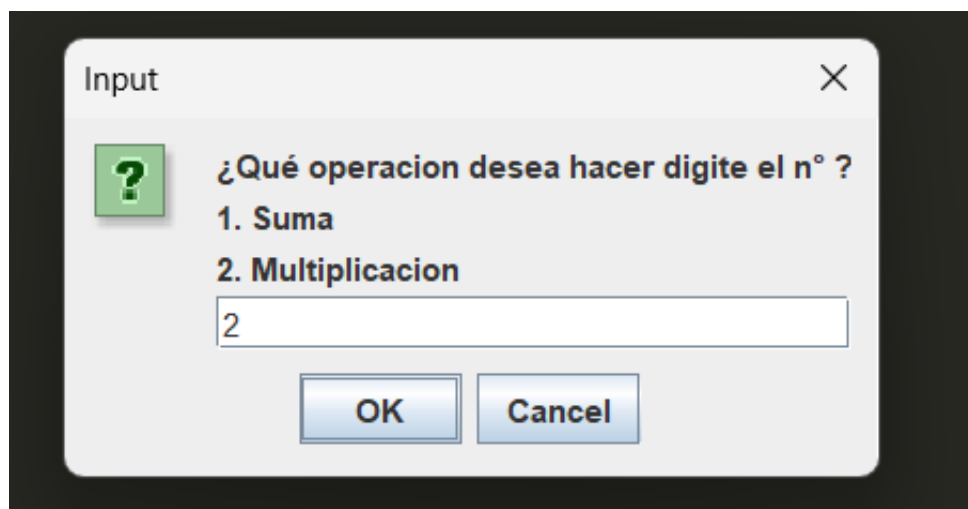
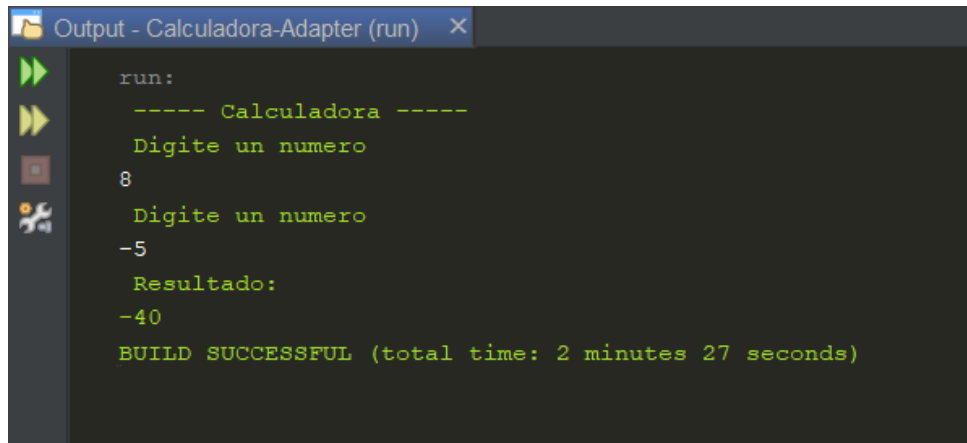


Figura 3: Salida 2



```
run:
----- Calculadora -----
Digite un numero
8
Digite un numero
-5
Resultado:
-40
BUILD SUCCESSFUL (total time: 2 minutes 27 seconds)
```

Figura 4: Salida de la ejecución mostrando la captura y conversión de datos mediante el Adaptador.

Repositorio

Enlace GitHub

- <https://github.com/Thalyhx/Modelos-de-Programacion/tree/main/Patrones%20Estructura/Calculadora-Adapter>

Estructura de Paquetes

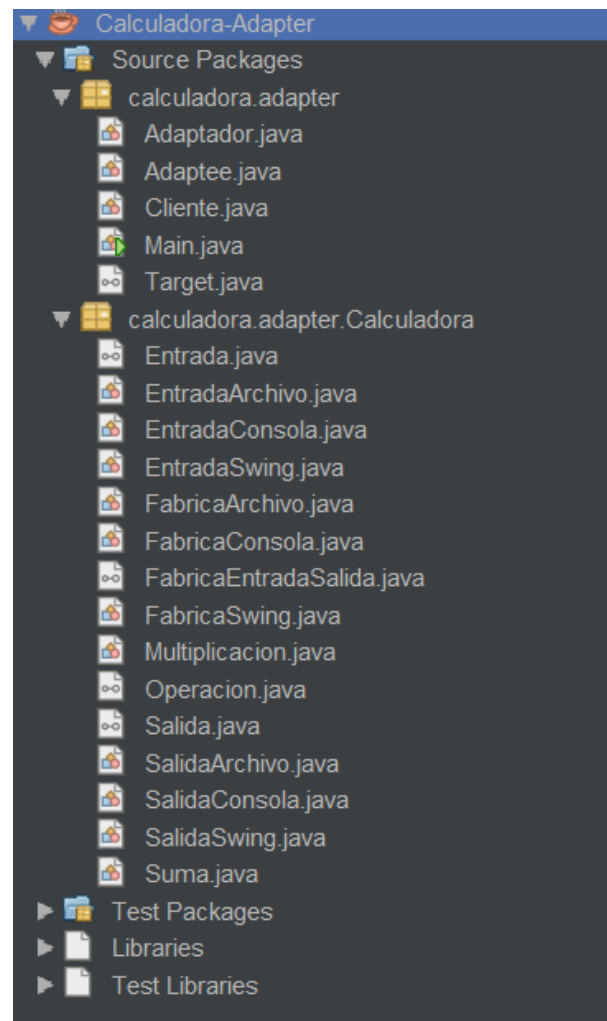


Figura 5: Árbol del proyecto en el IDE.